Appendix C

Improving a Unity Game with AI Assistance

In *Chapter 18*, *Hand-Crafting a 3D Game in Unity*, you built the first playable version of Crystal Cavern Escape. The result was intentionally simple, but it had the main pieces of a complete game: a level, player movement, a following camera, collectible crystals, hazards, a locked exit, win and lose conditions, restart behavior, and a standalone desktop build.

This appendix has a narrower purpose than *Chapter 18*: to show how AI can help improve a working Unity project without taking control away from you.

By the end of this appendix, you will have a slightly improved version of your game and, more importantly, a repeatable process for asking AI to help with Unity projects safely.

In this appendix, we will cover the following topics:

- Using AI after the game already works
- Choosing a low-cost AI workflow
- Adding one small feature safely
- Making the game look better with simple polish



Suggested chatbot prompts with example responses for this appendix can be found at the following link: https://github.com/markjprice/c-cpp-cs/blob/main/prompts/appendix-c.md

Using AI after the game already works

In *Chapter 18*, you built the first playable version of Crystal Cavern Escape. It was not a polished commercial game, and it was not supposed to be. It was a working baseline: the player could move, the camera could follow, crystals could be collected, hazards could end the game, the exit could unlock, and the project could be built as a desktop application.

That working baseline changes how you can use AI.

If you ask an AI assistant to create a Unity game from nothing, you are asking it to make many decisions at once: project structure, scene setup, player movement, input, camera behavior, collision rules, UI, assets, materials, win conditions, and build settings. Even if it produces something impressive, it may also produce a project you do not understand. When it breaks, you might not know whether the problem is in the code, the scene, the components, the packages, or the instructions you gave.

A working project gives you a safer starting point. You can now ask for smaller kinds of help. Instead of asking AI to invent the game, you can ask it to review what already exists, explain errors, suggest improvements, or help implement one focused change. That makes AI a sidekick rather than an automatic programmer.

Why a working baseline matters

A Unity project is not just a collection of C# files. The scripts matter, but so do the scene, GameObjects, components, materials, prefabs, UI objects, input package, and build profile. A chatbot can read and explain code easily, but it cannot always see how your scene is configured. A coding agent might be able to inspect files, but Unity scenes and prefabs are stored in formats that are not beginner-friendly to edit by hand.

This is why *Chapter 18* asked you to build the project step by step before using AI heavily. You now know the purpose of the main scripts:

- `PlayerController` moves the player.
- `CameraFollow` follows the player.
- `Collectible` detects collected crystals.
- `GameManager` stores game state and updates the HUD.
- `HazardZone` ends the game when the player touches danger.
- `ExitGate` checks whether the player can win.
- `PatrolMover` moves a simple hazard between two points.

You also know the important scene objects: `Player`, Main Camera, `GameManager`, HUDCanvas, `ExitGate`, the crystal prefab, the hazard zones, and the `Level` parent object.

That knowledge lets you judge AI suggestions. If AI says to put score logic into every crystal, you can recognize that this would duplicate work already handled by `GameManager`. If AI suggests using a Rigidbody for the player, you can ask why that is better than the existing Character Controller before changing anything. If AI suggests editing scene files directly, you can reject that approach and make changes in the Unity editor instead.



Prompt: Please explain why it is safer to use AI after a Unity game already works instead of asking AI to create the whole game from nothing.

Asking for advice before asking for code

One of the safest ways to use AI is to ask for ideas before asking for implementation. A beginner mistake is to ask for code too early. Code feels productive, but it can also hide design decisions that you have not thought through.

For example, this is a risky prompt:

```
Add power-ups to my Unity game.
```

That sounds clear, but it leaves too many decisions open. What kind of power-ups? How many? Do they need UI? Do they affect movement, hazards, time, or score? Should they be prefabs? Should they disappear after use? Should they stack? Should they have sound effects? Should old scripts be changed?

A better first prompt is:

```
Here is a summary of my Unity game. Suggest three small power-up ideas suitable
for a beginner project. Each idea must require no animation, no imported
assets, no scene file editing, and no more than one new C# script. Do not write
code yet.
```

This prompt asks AI to think with you before acting for you. It also sets constraints. Constraints are useful because they stop AI from expanding a small improvement into a much larger system.

After you choose one idea, you can ask for an implementation plan:

```
Create a short implementation plan for adding a speed boost pickup to this
game. List which existing scripts need to change, which new script should be
added, and which Unity editor steps are needed. Do not write code yet.
```

Only after reviewing the plan should you ask for code.



Good practice: Before asking AI to write code, ask for a short implementation plan. Check which scripts will change, which scene objects are involved, and how you will test the result.

Keeping changes small

When working with AI, small changes are safer than large changes. A small change has fewer places to go wrong and is easier to test. A large change might touch several scripts, require new prefabs, alter UI, change scene objects, and introduce new assumptions about the project.

For this appendix, treat every AI-assisted improvement as a controlled experiment. The workflow is:

1. Start from a working version.
2. Ask for one small improvement.
3. Review the explanation.
4. Apply the change manually or with a coding agent.

5. Test the game in Play mode.
6. Fix problems before adding anything else.
7. Keep or undo the change.

This is the same basic discipline you use in ordinary programming. AI does not remove the need for careful steps. It makes careful steps more important, because AI can produce plausible code that is still wrong for your project.

A small improvement might be:

- Add a speed boost pickup.
- Make hazards pause after the game ends.
- Display a better message when the exit is locked.
- Add a sound effect when a crystal is collected.
- Make the exit glow after it unlocks.
- Ask AI to review the scripts for readability.
- Ask AI to suggest better variable names or comments.



Good practice: Do not ask AI to review and rewrite the whole project. Review one script, choose one suggestion, make one change, and test.

Protecting the working version

Before using AI to change a project, protect the version that works. At minimum, make a copy of the project folder. Make a manual backup folder before changing anything. For example:

```
CrystalCavernEscape-before-ai-changes
```

This is not elegant, but it is better than losing the first working game.

What AI should and should not change

For this appendix, AI should mostly help with scripts, explanations, ideas, and checklists. Be careful about allowing AI to edit Unity-generated files.

Good AI-assisted tasks include:

- Explain a script.
- Review a script for readability.
- Suggest a small feature.
- Generate a first draft of a small script.
- Explain a Unity error message.
- Suggest Inspector values to try.

- Suggest visual polish that uses existing Unity features.
- Create a testing checklist.

Riskier AI-assisted tasks include:

- Editing `.unity` scene files directly.
- Editing `.prefab` files directly.
- Changing package settings.
- Replacing the player movement system.
- Changing many scripts at once.
- Adding packages without explaining why.
- Generating code that depends on newer C# features than Unity supports.

Coding agents need boundaries. A coding agent is most useful when you tell it exactly which files it may inspect, which files it may edit, and what kind of change you want. A chatbot is often safer for beginners because it gives advice that you can review before touching the project.



Good practice: Tell coding agents which files they may inspect and edit. For this project, start with `Assets/Scripts` only. Avoid direct edits to scene, prefab, `.meta`, package, and project settings files.

AI as reviewer, coach, and variation partner

In this appendix, AI will have three useful roles.

As a reviewer, AI can inspect the code and point out problems, such as repeated logic, unclear names, missing null checks, or methods that do too much. You should not accept every suggestion automatically, but review comments can help you notice details you missed.

As a coach, AI can explain Unity concepts in different ways. If you do not understand why a trigger collider is needed, why Time.deltaTime appears in movement code, or why the camera uses LateUpdate, you can ask for a simpler explanation or an analogy.

As a variation partner, AI can suggest controlled ways to change the game. A variation is not a complete rewrite. It is a small, testable change to an existing design. For example, “add one speed boost pickup” is a variation. “Turn this into a full dungeon crawler” is a new project.

This is the healthiest role for AI in a beginner programming book. You remain the developer. AI helps you think, review, and experiment, but you decide what belongs in the project.

Now that you know why AI is safer after the game already works, you can choose a workflow that matches your budget and tools. In the next section, you will compare a low-cost chatbot-first approach with optional coding-agent workflows, so you can use AI without assuming an expensive subscription.



Prompt: Please explain the difference between using AI as a reviewer, coach, and variation partner in a beginner Unity project.

Choosing a low-cost AI workflow

You do not need an expensive AI subscription to improve Crystal Cavern Escape. For this appendix, the default workflow is simple: use a chatbot for advice, explanations, reviews, and small code suggestions, then apply the changes yourself in Unity and your code editor.

A chatbot-first workflow is the safest and cheapest option:

1. Describe the current project.
2. Paste one script or one error message.
3. Ask for advice before asking for code.
4. Make the change yourself.
5. Test the game in Unity.
6. Keep or undo the change.

This works with free or low-cost chatbots such as ChatGPT, Gemini, Claude, or Grok. The exact limits and prices change, so do not build your learning process around one paid tool. The skill you are learning is how to ask focused questions and evaluate the answers.

Chatbot-first workflow

Use a chatbot when you want to understand or plan.

Good chatbot tasks include:

- Explain a Unity error.
- Review one C# script.
- Suggest three small improvements.
- Explain why a trigger collider is not working.
- Suggest Inspector values to try.
- Create a testing checklist.
- Draft a small script for you to review.

For example:

```
I am building a beginner Unity game called Crystal Cavern Escape. The player
collects six crystals, avoids hazards, and reaches an exit gate. Review this
script for readability and beginner-friendly Unity style. Do not rewrite it
yet. First, explain any problems you see.
```

This kind of prompt keeps you in control. The chatbot reviews and explains. You decide what to change.

Coding-agent workflow

A coding agent can inspect and edit project files directly. Examples include OpenCode, Claude Code, and OpenAI Codex. These tools can be useful, but they should be given narrow tasks.

Use a coding agent only after you have a working backup or source control commit. Limit the task to scripts first. Avoid letting the agent edit Unity scene files, prefab files, package files, or project settings unless you understand the change.

A safe coding-agent prompt is:

```
Inspect only the C# scripts in Assets/Scripts. Do not edit scene files, prefab
files, package files, meta files, or project settings. Suggest one small
readability improvement. Wait for approval before changing anything.
```



Prompt: Please explain why editing Unity `.unity`, `.prefab`, `.meta`, package, or project settings files directly can be risky for beginners. Create a safe prompt for a coding agent that may inspect only `Assets/Scripts` and must not edit scene, prefab, package, `.meta`, or project settings files.

Choosing what to ask for

The best AI tasks in this appendix are small and specific. For example:

```
Suggest one beginner-friendly feature that can be added to this Unity game with
one new script and small changes to PlayerController.
```

or:

```
Explain how to add a temporary speed boost pickup without changing the existing
win, lose, hazard, or exit logic.
```

Avoid broad requests that would turn this appendix into a new project.

Now that you have chosen a low-cost workflow, the next step is to give AI enough project context to be useful. In the next section, you will write a short Unity project brief that explains the game, the scripts, and the boundaries for safe AI help.

Writing a Unity project brief for AI

AI gives better answers when it understands the project. Instead of pasting random scripts into a chatbot and hoping it guesses the context, write a short project brief. This gives the AI assistant the same starting point each time.

Create a file in the project folder named `AI_PROJECT_BRIEF.md`. You do not need to place it inside the `Assets` folder because Unity does not use it. It is for you and your AI assistant.

Add the following content:

```
# Crystal Cavern Escape project brief

This is a beginner Unity 6 project using C#.

The game is a small 3D collect-and-escape game:
- The player moves around a simple 3D level.
- The player collects six crystals.
- Hazards cause the player to lose.
- The exit gate unlocks after all crystals are collected.
- Reaching the unlocked exit wins the game.
- Pressing R restarts after winning or losing.

Main scripts:
- PlayerController: moves the player using a Character Controller.
- CameraFollow: follows the player from above and behind.
- Collectible: detects when the player collects a crystal.
- GameManager: stores score and game state, updates HUD text, handles win/loss/
restart.
- HazardZone: causes a loss when the player enters a hazard trigger.
- ExitGate: checks whether the exit is unlocked and changes gate appearance.
- PatrolMover: moves a hazard between two points.

Important scene objects:
- Player
- Main Camera
- GameManager
- HUDCanvas
- ExitGate
- Crystal prefab
- Hazard zones
- PatrolHazard
- Level

Rules for AI help:
- Use Unity-compatible C#.
- Keep the code beginner-friendly.
- Prefer small changes.
- Explain the plan before writing code.
- Do not rewrite the whole project.
```

```
- Do not change scene files, prefab files, package files, .meta files, or
project settings unless asked.
- Do not add paid assets, complex animation, multiplayer, pathfinding,
inventory, or save/load systems.
```

You can add this brief file into a chatbot before asking for help. If you use a coding agent, point it to this file before asking it to inspect the project.

A good first prompt is:

```
Read this Unity project brief. Do not write code yet. Summarize how the project
works and suggest three small improvements that would be suitable for a
beginner C# project.
```

This prompt asks for understanding before implementation. If the AI summary is wrong, correct it before continuing. If the summary is accurate, you can ask for a more focused plan.

For example:

```
Suggest a plan for adding one speed boost pickup. The change should use one new
script and small changes to existing scripts. Do not write code yet.
```

A project brief is useful because it reduces guesswork. It tells AI what exists, what should stay simple, and what it must not change. It also helps you think clearly about the project before requesting code.

Now that the AI assistant has a clear summary of the project, you can use it for a safer first task: reviewing the existing C# scripts before adding new features.

Asking AI to review the scripts

Before asking AI to add new code, ask it to review the existing scripts. A review is safer than a rewrite because the game already works. The aim is to find small improvements without changing behavior.

Start with one script at a time. Link to the project brief first, then add a script such as `PlayerController`, `GameManager`, or `ExitGate`.

Use a prompt like this:

```
Here is my Unity project brief and one C# script. Review the script for
beginner-friendly Unity style. Look for unclear names, repeated logic, missing
null checks, and code that might be hard to extend. Do not rewrite the script
yet. List only the three most useful improvements.
```

This prompt gives the AI a narrow task. It asks for review comments, not a new version of the whole script.

Good review suggestions might include:

- Rename a field or method to make its purpose clearer.
- Add a missing null check.
- Move repeated logic into a helper method.

- Replace a magic number with a serialized field.
- Add a short comment where Unity behavior might surprise beginners.

Be careful with suggestions that change the project design. For example, AI might suggest replacing the `CharacterController` with Rigidbody physics, using events instead of direct method calls, or splitting `GameManager` into several classes. Those ideas might be valid in a larger project, but they are too disruptive here.

After reading the review, choose one small improvement. Then ask:

```
Show the smallest safe code change for the first suggestion only. Preserve the
current behavior. Explain what changed in two sentences.
```

Apply the change, return to Unity, and test the game. If the game still works, keep the change. If it breaks, undo it and ask the AI to explain the problem.

A useful review process is:

1. Review one script.
2. Choose one suggestion.
3. Make one change.
4. Test the game.
5. Keep or undo the change.

Do not ask AI to review and rewrite every script at once. That makes too many changes to test. Small reviews teach you more and keep the project stable.

Once the scripts have been reviewed, you can use the same careful approach to add one visible improvement. In the next section, you will add a small feature without redesigning the game.

Adding one small feature safely

Now use AI to add one small feature: a speed boost pickup. When the player touches it, movement speed increases for a few seconds. This is a good AI-assisted change because it is visible, easy to test, and does not affect scoring, hazards, the exit gate, or restart logic.

Start by asking AI for a plan, not code:

```
I have a beginner Unity game with a PlayerController script that moves the
player using a Character Controller. Suggest the smallest safe plan for adding
a temporary speed boost pickup. Use one new script and a small change to
PlayerController. Do not write code yet.
```

Review the answer. It should suggest two changes:

1. Add a method to `PlayerController` that temporarily changes movement speed.
2. Add a new `SpeedBoostPickup` script that detects the player, applies the boost, and destroys the pickup.



Good practice: Keep a short `AI_NOTES.md` file that lists what AI helped with and what you reviewed and tested yourself. This is especially useful in education.

Updating PlayerController

Open `PlayerController.cs`. Add this import at the top:

```
using System.Collections;
```

Add these fields:

```
private float speedMultiplier = 1f;
private Coroutine speedBoostCoroutine;
```

Change the movement line in `Update` from:

```
controller.Move(movement * moveSpeed * Time.deltaTime);
```

to:

```
controller.Move(movement * moveSpeed * speedMultiplier * Time.deltaTime);
```

Add this method to the class:

```
public void ApplySpeedBoost(float multiplier, float duration)
{
    if (speedBoostCoroutine is not null)
    {
        StopCoroutine(speedBoostCoroutine);
    }

    speedBoostCoroutine = StartCoroutine(
        SpeedBoostRoutine(multiplier, duration));
}

private IEnumerator SpeedBoostRoutine(float multiplier, float duration)
{
    speedMultiplier = multiplier;

    yield return new WaitForSeconds(duration);

    speedMultiplier = 1f;
    speedBoostCoroutine = null;
}
```

This keeps the change inside `PlayerController`, where movement speed already belongs.



Prompt: Please explain why `StopCoroutine` might be useful when the player collects another speed boost before the first one expires.

Creating the speed boost pickup

Create a new script named `SpeedBoostPickup` in the `Scripts` folder:

```
using UnityEngine;

public class SpeedBoostPickup : MonoBehaviour
{
    [SerializeField] private float multiplier = 1.75f;
    [SerializeField] private float duration = 4f;

    private void OnTriggerEnter(Collider other)
    {
        if (!other.TryGetComponent<PlayerController>(
            out PlayerController player))
        {
            return;
        }

        player.ApplySpeedBoost(multiplier, duration);
        Destroy(gameObject);
    }
}
```

This follows the same trigger pattern as the collectible crystals. The pickup checks whether the object entering the trigger is the player, applies the effect, then removes itself.

Adding the pickup to the scene

In Unity, create the pickup:

1. Create a sphere in the level.
2. Rename it to `SpeedBoostPickup`.
3. Set its position to `0`, `0.5`, `-4`.
4. Set its scale to `0.8`, `0.8`, `0.8`.
5. Create a material named `Mat_SpeedBoost`.
6. Give it a bright color that differs from crystals and hazards.

7. Assign the material to the sphere.
8. Enable **Is Trigger** on its **Sphere Collider**.
9. Attach the `SpeedBoostPickup` script.

Press **Play** and test it. Walk normally, collect the pickup, and confirm that the player moves faster for a few seconds. Then check that crystals, hazards, the exit, win, lose, and restart still work.

Asking AI to check the change

After testing, ask AI to review only the change:

```
Review this small Unity change. PlayerController now has a temporary speed
boost method, and SpeedBoostPickup calls it from OnTriggerEnter. Check for
beginner-level problems only. Do not redesign the feature.
```

Do not accept suggestions that turn this into a larger power-up system. The aim is one safe improvement, not a new framework.

You have now used AI in a controlled way: plan first, change one feature, test the result, and reject unnecessary expansion.



Prompt: Suggest three small improvements for a beginner Unity collect-and-escape game. Each improvement must require no imported assets, no animation, and no more than one new script.

In the next section, you will use AI for a different kind of improvement: making the game look better without adding complex assets or systems.

Making the game look better with simple polish

A small Unity game does not need expensive assets to look better. It needs clear visual choices. The player should be able to tell what is safe, what is dangerous, what can be collected, and where to go next.

Use AI as an art-direction assistant, not as a replacement for the Unity editor. Ask it for a simple style guide:

```
Create a simple visual style guide for my beginner Unity game Crystal Cavern
Escape. The game must use only primitive shapes, basic materials, lights,
TextMeshPro UI, and simple particles. Suggest colors for the floor, walls,
crystals, hazards, exit gate, and speed boost pickup. Do not suggest imported
models, paid assets, custom shaders, or animation.
```

Review the answer and choose a small set of colors. Do not keep changing everything. A limited palette usually looks better than many unrelated colors.



Good practice: Visual polish should make the game easier to understand. Crystals, hazards, the exit, and the speed boost pickup should each have a distinct visual identity.

Improving materials

Create or update these materials:

- `Mat_Floor`: Dark stone or muted gray.
- `Mat_Wall`: Darker rock color.
- `Mat_Crystal`: Bright blue, purple, or cyan.
- `Mat_Hazard`: Red or orange warning color.
- `Mat_ExitLocked`: Dull gray or dark red.
- `Mat_ExitUnlocked`: Bright green, gold, or cyan.
- `Mat_SpeedBoost`: A color that is not used by crystals or hazards.

Use strong contrast. The player should never confuse a collectible with a hazard.

If you are using URP, you can make crystals and the unlocked exit feel brighter by enabling emission on their materials. Keep it subtle. The aim is readability, not visual noise.

Improving lighting

Lighting can change the mood of the whole scene quickly. For a crystal cave, use a darker environment with a few bright objects.

Try these simple changes:

- Lower the intensity of the main directional light.
- Add one or two point lights near the crystals or exit.
- Make the exit area brighter than the rest of the level.
- Avoid making hazards too dark to see.

Ask AI for suggested starting values:

```
Suggest simple Unity lighting values for a small low-poly crystal cave. Use one
Directional Light and two Point Lights. Keep the result readable for a beginner
game.
```

Treat the values as starting points. Test in the Game window and adjust by eye.

Adding simple particles

A particle effect can make an object feel active. Do not add particles everywhere. Add them only where they communicate useful information.

Good places for particles are:

- A soft sparkle around crystals.
- A warning effect above hazards.
- A glow or burst near the unlocked exit.
- A short effect when the speed boost pickup is collected.

For a quick improvement, add a **Particle System** as a child of the `ExitGate`. Keep the particles small, slow, and bright. Enable it only when the exit is unlocked if you want an extra challenge, or leave it always on if you want to keep this section simple.

Ask AI for settings rather than code:

```
Suggest beginner-friendly Unity Particle System settings for a glowing exit
gate. Use only built-in particle settings. Keep the effect subtle and easy to
read.
```

Improving the HUD

The HUD should be easy to read while the player is moving. Increase contrast and reduce clutter.

Check that:

- The crystal counter is large enough.
- The message text is not hidden by the scene.
- Win and lose messages are clear.
- The speed boost does not require extra UI unless you want it.

A simple improvement is to add a dark semi-transparent background behind the HUD text. This often makes UI text easier to read than changing the text color repeatedly.

Ask AI for a short UI review:

```
Review this HUD for a beginner Unity game: crystal counter in the top-left and
message text near the top-center. Suggest three simple readability improvements
using TextMeshPro and basic UI objects only.
```

Keeping polish controlled

After each visual change, play the game. Check that the new look does not hide gameplay information.

Use this checklist:

- Crystals are easy to see.
- Hazards look dangerous.
- The exit looks different when locked and unlocked.
- The speed boost pickup has its own visual identity.
- The HUD remains readable.
- The scene still runs smoothly.

- The game rules are clearer than before.

Visual polish should support the game. If an effect looks nice but makes the game harder to understand, remove it.



Prompt: Suggest a simple visual style guide for a low-poly Unity crystal cave using only primitive shapes, basic materials, lights, TextMeshPro UI, and simple particles.



Good practice: After testing AI-assisted changes, create a new build such as `CrystalCavernEscape-v0.2`. Keep the original build so readers can compare the plain version with the improved version.

You have now used AI to improve the game's presentation without replacing the project or adding complex systems.

Practice, references, and resources

In this section, you will be tested on what you've learned in this appendix and be provided with useful links to reference materials and resources.

Check your understanding

Answer the following questions about the topics we covered in this appendix:

1. Why should AI be used after the Unity game already works?
2. What is the risk of asking AI to "make my Unity game better"?
3. Why is a chatbot-first workflow suitable for students?
4. What is the purpose of `AI_PROJECT_BRIEF.md`?
5. Why should a coding agent be told which files it may and may not edit?
6. Why should AI review one script at a time?
7. What does the speed boost pickup do?
8. Why does the speed boost feature belong mostly in `PlayerController`?
9. Why should visual polish use clear contrast between crystals, hazards, the exit, and pickups?
10. What should you test after adding an AI-assisted change?



Answers are found in the GitHub repository for this book at the following link:

`https://github.com/markjprice/c-cpp-cs/blob/main/docs/Appendix-A.pdf`

Links

ChatGPT Pricing: `https://chatgpt.com/pricing/`

OpenCode, the open source AI coding agent: `https://opencode.ai/`

Particle System component reference:

`https://docs.unity3d.com/6000.3/Documentation/Manual/class-ParticleSystem.html`

Particle effects:

`https://docs.unity3d.com/6000.3/Documentation/Manual/ParticleSystems.html`

Summary

In this appendix, you learned how to:

- Use AI after you already have a working Unity game.
- Treat AI as a reviewer, coach, and variation partner.
- Choose a low-cost chatbot-first workflow.
- Give coding agents narrow boundaries.
- Write an `AI_PROJECT_BRIEF.md` file for a Unity project.
- Ask AI to review scripts before generating new code.
- Add one small AI-assisted feature: a speed boost pickup.
- Keep the speed boost change focused on `PlayerController` and one new pickup script.
- Use AI to suggest simple materials, lighting, particles, and HUD polish.
- Test AI-assisted changes before keeping them.

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow this QR code:

`https://packt.link/HdyY4`

